PHYS 3810: full-course student setup

REVIEW DRAFT · Document v0.1 · Proposed course code v0.2.0-rc1

The full-course download is **unpublished**. Use the complete course folder your instructor approves. The earlier Grover-only package does not contain all five algorithm lessons. Session dates, classroom coverage, and assignments remain to be confirmed.

These instructions use **64-bit Linux and CPython 3.12**, with Qiskit 2.6.1 and Aer 0.17.2. Recorded setup and browser checks passed on Ubuntu 26.04.1 LTS x86_64, Python 3.12.15, and JupyterLab 4.6.4, including a fresh application environment and the in-folder kernel setup below. The walkthrough used isolated Jupyter settings and a manually opened local URL; automatic browser opening was not checked.

Native Windows x64, macOS ARM64/Intel, and VS Code/Cursor notebook setup are **untested**. Windows ARM64 and 32-bit Python are outside this draft's target. Ask the instructor for an approved setup if needed. Git, IBM accounts, tokens, GPUs, and quantum hardware are not required.

1. Open the complete course folder

Extract the approved folder completely and keep its files together. Individual downloaded notebooks cannot find the shared course code on their own. Open a Linux terminal in the folder containing `requirements.txt`, `constraints-ci.txt`, `src/`, `scripts/`, `tests/`, and `teaching/`. This is the **course root**, not the `teaching/` subfolder.

Replace the quoted path with your actual folder:

```
cd "/path/to/phys3810-fa2026"
ls
python3.12 --version
```

Expected: `Python 3.12.x`. If `python3.12` is missing, or this is a managed computer, obtain the instructor-approved Python 3.12 installation before continuing. Do not change system security policy or install the course packages into the operating system's Python.

Run this guide's commands in the **terminal**, not at a Python `>>>` prompt or in a notebook cell. Installation needs internet access; default checks and lessons run locally afterward.

2. Create a separate Python environment

Create the environment once in your course folder:

```
python3.12 -m venv .venv
source .venv/bin/activate
python --version
python -c "import sys; print(sys.executable)"
```

The version must be 3.12 and the printed interpreter must end in this course folder's `.venv/bin/python`. This environment keeps the course packages separate from other projects. If `.venv` belongs to another project or Python version, ask for help before reusing it.

3. Install the specified packages and check the lessons

With `.venv` activated, run:

```
python -m pip install --upgrade pip==26.2.2
python -m pip install -r requirements.txt
python scripts/validate_classroom.py
```

`requirements.txt` includes JupyterLab and the circuit, plotting, widget, and notebook-execution libraries. It automatically reads `constraints-ci.txt` to select the specified package versions. Keep both files unchanged; do not upgrade individual packages during the lessons.

The validator prints your interpreter path and expected Qiskit/Aer versions, checks package consistency, runs **160 tests**, and executes **eight notebooks in separate fresh kernels**. The current lesson set has **50 nonempty code cells** and produces **16 PNG figures**. Success ends with:

```
PASSED: classroom validation
```

`SKIPPED: optional IBM Runtime and GPU execution` is expected. A `FAILED` line or traceback needs attention; do not bypass a failing cell. Executed copies go to `artifacts/executed-classroom/`; source notebooks are not overwritten.

4. Open JupyterLab and select the class kernel

Run these commands in the same activated terminal:

```
python -m ipykernel install --sys-prefix --name phys3810-fa2026 --display-name "Python (PHYS 3810)"
python -m jupyterlab
```

Keep the terminal open. If the browser does not open, use the local URL printed by JupyterLab. In its file browser, open:

`teaching/phys3810-fa2026/notebooks/00_setup_preflight.ipynb`

1. Choose **Kernel → Change Kernel → Python (PHYS 3810)**.
2. Choose **Kernel → Restart Kernel and Run All Cells**; confirm the restart.
3. Compare the notebook's printed interpreter with the terminal path from step 2. Both must point to the same course `.venv/bin/python`. A kernel's display name alone is not proof of its interpreter.
4. Check that every code cell finishes without errors and preflight shows a circuit drawing and measurement histogram. The notebook checks its numerical results; sampled counts need not match another computer exactly.
5. Save with **File → Save Notebook**.

Use the same registered environment for other lessons, with a separate kernel for each notebook. Start at the top. If you edit a parameter, rerun the calculations and plots that depend on it. Restart/run-all helps catch values left over from an earlier run.

5. Find the notebooks

These are paths **inside the complete course folder**, not online download links. The six course notebooks are under:

`teaching/phys3810-fa2026/notebooks/`

Notebook	What it covers
`00_setup_preflight.ipynb`	Environment, a simple circuit, sampling, and figures.
`01_grover.ipynb`	Phase marking and amplitude amplification on 2–5 search qubits.
`02_qft.ipynb`	Exact Fourier transforms on 1–6 qubits, including complex phase and bit ordering.
`03_qpe.ipynb`	Phase estimation with 1–6 counting qubits and a unitary on **one or two target qubits**.
`04_hhl.ipynb`	A positive Hermitian 2×2 system with exact integer eigenvalues 1–7. The output is a normalized, postselected solution state; this is not a general linear-system solver.
`05_shor.ipynb`	A separate compiled **N=15** example with **four residue qubits** and 4–6 counting qubits, followed by classical order/factor recovery. It is not scalable general factoring.

Two additional checks live under the root `notebooks/` directory:

Notebook	What it checks
`00_local_statevector_smoke.ipynb`	Exact local simulation.
`01_local_aer_smoke.ipynb`	Local Aer sampling and a small noise example.

The full validator runs all eight notebooks. Availability does not determine what is taught or submitted; follow your instructor's eventual lesson and assessment instructions.

Use predictions during the lessons

Before a lesson, run all cells to check setup. During discussion, restart the kernel and clear displayed outputs, make the requested predictions, then run the relevant cells. The instructor will identify which exercises to discuss. All required code still runs during the setup check.

The notebooks already contain worked calculations, assertions, and reference results. Follow the instructor's directions about collaboration, solutions, and any assessed work.

6. Finish and return later

Save the notebooks you edited. Choose **File → Shut Down** in JupyterLab, confirm, and wait for the server-stopped message before closing the browser tab. Closing a tab alone does not stop its server or kernels.

Next time, open a terminal in the same course root:

```
source .venv/bin/activate
python -c "import sys; print(sys.executable)"
python -m jupyterlab
```

Reopen preflight, select **Python (PHYS 3810)**, and restart/run all to check the interpreter and figures. Installation and registration normally happen once for an environment that stays in the same location. Keep your edited notebooks separate from a fresh supplied copy so you can compare or restore the original exercises.

If something fails

- **Missing file:** check `pwd` and `ls`. Work in the complete course root, not inside the ZIP or `teaching/`.
- **Wrong Python or missing package:** print `sys.executable` again. Activate the course environment, or use `.venv/bin/python` instead of `python` in terminal commands. Check the notebook kernel separately.
- **Terminal passes, notebook fails:** select the class kernel, compare interpreter paths, restart it, and run all cells from the top.
- **Kernel absent or points elsewhere:** repeat step 4's registration command in the intended activated environment, relaunch JupyterLab, and select the kernel again.
- **Plot or circuit drawing fails:** use the notebook's Python environment to rerun `python -m pip install -r requirements.txt`, then restart its kernel.
- **Installation asks for a compiler, fails, or cannot create `venv`:** save the error and ask for the approved setup. Do not bypass pins or mix in another Qiskit installation.
- **Folder or environment moved:** preserve your notebook work, then create a fresh environment at the new location, reinstall the specified packages, and register its kernel. Virtual environments are not portable; re-registering alone does not repair a moved environment.

For help, provide the failing command, operating system, Python version, interpreter path, and error text. Omit credentials, private account details, other students' information, and the token-bearing JupyterLab URL.

Official reference links

- Python 3.12 virtual environments
- IPython kernel registration
- Starting JupyterLab

These references explain the tools. Follow the course commands and pinned versions above for this draft.